

**Факультет Программной инженерии и компьютерной техники
Образовательная программа Проектирование и разработка систем больших данных
Направление подготовки (специальность) 09.04.04 - Программная инженерия**

ОТЧЕТ

по лабораторной работе №4
по курсу «Структуры и алгоритмы в базах данных и распределенных системах»
на тему: «Thread-safe хеш-таблица с закрытой адресацией.»

Обучающийся: Никифорова Е. А., Р4135

Преподаватель: Платонов А. В., доцент

Преподаватель: Портнов П. В., ассистент

Содержание

Задание	3
Concurrent hash-map	3
Реализация	4
Архитектура и синхронизация MyConcurrentHashMap	4
1. Структура данных	4
2. Схема синхронизации	4
3. Алгоритмы записи (CAS-логика)	4
4. Итератор	4
Тестирование и результаты	6
1. Тестирование многопоточности (JCStress)	6
2. Производительность (JMH)	6
Выводы по графикам:	6

Задание

Concurrent hash-map

Требуется реализовать thread-safe хеш-таблицу с закрытой адресацией.

Требования к структуре:

- минимально необходимый набор операций:
 1. put(key: K, value: V),
 2. get(key: K) -> V,
 3. size() -> usize,
 4. clear(),
 5. merge(key: K, value: V, merger: Fn(V, V) -> V) -> V,
 6. итератор по парам ключ-значения
- (почти) никогда не блокирующие операции чтения
- однозначный наблюдаемый порядок между завершёнными операциями
- За более формальной спецификацией см. Javadoc к ConcurrentHashMap (<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>) в JDK.
- При этом не требуется реализовывать конкретный интерфейс для таблицы из выбранного ЯП, это просто референс.

В работе также требуется:

- Написать бенчмарки (сравнивать перф уместно с не-thread-safe версией)
- Написать concurrency-тесты с использованием специализированного инструментария (например, если Java, jstress)
- Нарисовать графики по числовым результатам
- Объяснить интересные результаты в отчёте

Реализация

Архитектура и синхронизация MyConcurrentHashMap

1. Структура данных

- **Массив бакетов (table):**
 - Представляет собой `volatile` поле типа `AtomicReferenceArray`.
 - Это обеспечивает безопасное чтение и изменение ячеек массива из разных потоков без блокировок.
- **Узел списка (Bucket):**
 - Хранит ключ `key`, ссылку на следующий элемент `next` и значение `value`.
 - Значение обернуто в `AtomicReference`, что позволяет выполнять безопасные атомарные обновления.
- **Счетчик размера (size):**
 - Основан на `LongAdder`.

2. Схема синхронизации

Для координации потоков применяется `ReentrantReadWriteLock`:

- **Чтение (get):**
 - Выполняется полностью без блокировок.
 - Потоки-читатели не конкурируют с пишущими потоками и не блокируют их.
- **Запись (put, merge):**
 - Потоки захватывают разделяемую блокировку чтения (`readLock`).
 - Это позволяет множеству потоков выполнять запись одновременно.
 - Конфликты на уровне конкретных бакетов разрешаются с помощью CAS.
- **Ресайз (resize):**
 - Запускается при заполнении таблицы на 50%.
 - Операция захватывает эксклюзивную блокировку записи (`writeLock`).

3. Алгоритмы записи (CAS-логика)

Добавление и обновление элементов на уровне конкретного бакета выполняется без блокировок в бесконечном цикле `while(true)`:

- **При обновлении значения существующего ключа:**
 - Применяется CAS-цикл над `AtomicReference` значения ноды.
- **При вставке нового ключа:** Новый узел создается локально со ссылкой на текущую голову бакета. Затем выполняется попытка CAS-замещения ячейки массива `AtomicReferenceArray`.

4. Итератор

- При создании итератора фиксируется снимок массива `table`
 - Это гарантирует отсутствие исключений при параллельных модификациях таблицы.

Тестирование и результаты

1. Тестирование многопоточности (JCStress)

Проверена корректность работы при одновременной записи ключей-коллизий (1 и 17 в бакет 1):

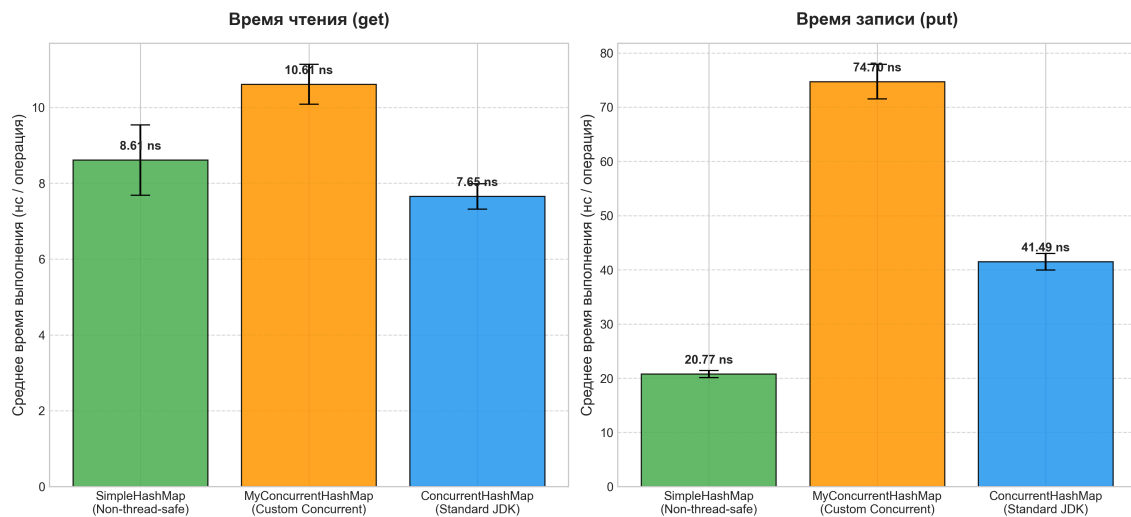
- **SimpleHashMap (сбой):** Стабильно теряет данные (Lost Update, состояние 1)
- **MyConcurrentHashMap (успех):** Прошла тест со статусом **Passed** (состояние 2 в 100% случаев)

Подробные результаты испытаний доступны в сгенерированных HTML-отчетах:

- [Детальный отчет по сбоям SimpleHashMap](#)
- [Детальный отчет по верификации MyConcurrentHashMap](#)

2. Производительность (JMH)

Измерено среднее время выполнения одной операции (Average Time) в однопоточном режиме в наносекундах:



Выводы по графикам:

- **Чтение (get):**

Все реализации показывают близкие результаты (разница в пределах 2-3 нс).

MyConcurrent работает на уровне неблокирующей за счет прямого чтения volatile-переменных без блокировок.

- **Запись (put):**

Кастомная мапа работает медленнее всех (74.70 нс). Расходы на захват `tableLock.readLock()` и аллокацию памяти под обертки `AtomicReference` для каждого узла. JDK-версия работает быстрее (41.49 нс).